



الجمهورية العربية السورية
جامعة دمشق
كلية الهندسة المعلوماتية

مشروع مادة البرمجة المتوازية

High-Performance E-Commerce Backend Engine

إعداد الطلاب:

محمد عامر هيثم الشعار

أديب عبد الرزاق الزين

إشراف:

المهندس حذيفة عقيل

• مقدمة عن إطار العمل Laravel Framework :

- تم بناء وتطوير هذا النظام بالاعتماد على إطار العمل Laravel، وهو أحد أقوى أطر عمل لغة PHP وهو مفتوح المصدر.
- قد صُمم Laravel لتطوير تطبيقات الويب المتقدمة باتباع نمط البنية الهندسية MVC (Model-View-Controller)، والذي يضمن فصلاً كاملاً بين المنطق البرمجي (Business Logic)، وهيكل البيانات (Data Models)، وواجهات المستخدم (Views).
- يتميز Laravel بتقديم "كتابة برمجية أنيقة (Elegant Syntax)" تقلل من تعقيد الأكواد التقليدية، مع توفير بيئة متكاملة تحتوي على أدوات مدمجة لمعالجة التحديات البرمجية المعقدة بفعالية وموثوقية عالية.

• Why Laravel ؟

- لم يكن اختيار Laravel عشوائياً، بل جاء استجابة لمتطلبات المشروع التقنية غير الوظيفية (Non-Functional Requirements)، وأهمها الأداء العالي والموثوقية.
 - تلخص الأسباب التالية سبب اختيار لارافيل كبيئة عمل لمشروعنا:
- ✓ التعامل مع العمليات المتزامنة – **Concurrency Handling**: يوفر إطار العمل أدوات متقدمة للتفاعل مع قاعدة البيانات تتيح تطبيق آليات القفل، مما يجعله الخيار الأمثل لأنظمة التجارة الإلكترونية التي تتطلب حماية صارمة ضد مشاكل Race Conditions أثناء خصم المخزون.

✓ **الاستقرار تحت الضغط – Scalability & Queues**: يدعم Laravel بشكل أصلي أنظمة الطوابير (Queues) ، مما سمح لنا بترحيل العمليات التي تستهلك موارد الخادم (مثل معالجة وإنشاء الفواتير والتقارير) إلى الخلفية، وبالتالي الحفاظ على سرعة استجابة النظام حتى في أوقات الذروة.

✓ **الأمان المدمج (Built-in Security)**: نظراً لتعامل النظام مع بيانات حساسة وفواتير يوفر Laravel حماية تلقائية ضد الثغرات الشائعة مثل حقن قواعد البيانات (SQL Injection) والهجمات عبر المواقع (XSS و CSRF) ، دون الحاجة لكتابة أكواد حماية معقدة من الصفر.

✓ **قابلية التوسع (Scalability)**: من أهم ميزات Laravel أنه مناسب للمشاريع الصغيرة والكبيرة في نفس الوقت فيمكن البدء بمشروع بسيط، وتطوره بدون مشاكل بدون الحاجة لإعادة بناء المشروع من الصفر.

✓ **التكامل مع التقنيات الحديثة**: يمكن لـ Laravel العمل بسهولة مع: Redis – AWS– RabbitMQ– Elasticsearch– WebSockets– Docker وهذا يجعله مناسباً لبناء أنظمة حديثة عالية الأداء.

✓ **دعم بناء (RESTful APIs)**: يُعتبر Laravel من أفضل الخيارات لبناء APIs الحديثة، لأنه يوفر: API Resources–Resource Controllers – Validation جاهز – Authentication باستخدام Sanctum أو Passport أو Jwt

✓ **نظام (Artisan CLI)**: يوفر Laravel أداة قوية اسمها Artisan تساعد على إنشاء أجزاء المشروع بسرعة كبيرة.

✓ **يقوم بإنشاء: model –controller –migration** في امر واحد وهذا يوفر وقتاً كبيراً أثناء التطوير.

✓ يحتوي إطار العمل على حزمة واسعة من الميزات، تم توظيف أبرزها لخدمة أهداف المشروع كما يلي:

← محرك قواعد البيانات (Eloquent ORM) : نظام متقدم للتعامل مع قواعد

البيانات بأسلوب كائني التوجه (Object-Oriented) وقد تم استغلال ميزة ال

Eager Loading لتحسين استعلامات قاعدة البيانات وحل مشكلة (N+1)

Queries) مما أدى إلى تسريع جلب بيانات سلة المشتريات بشكل كبير وتقليل العبء

على السيرفر.

← إدارة المعاملات (Database Transactions) : يضمن لنا هذا المفهوم تطبيق

مبدأ الذرية (Atomicity) ، بحيث يتم التراجع عن عملية الدفع بالكامل في حال حدوث

أي خطأ طارئ، مما يحمي قاعدة البيانات من التشوه والتخريب.

← نظام التوجيه والتحكم في التدفق (Routing & Middleware) : يوفر نظاماً

مرناً لإدارة مسارات التطبيق (Routes) وقد تم استخدام طبقة ال Middleware

لتطبيق مفهوم Rate Limiting، والذي يقوم بضبط عدد الطلبات المسموح بها لكل

مستخدم، مما شكل جدار حماية أولياً يمنع استنزاف موارد السيرفر (Resource

Exhaustion) نتيجة الطلبات الوهمية أو هجمات حجب الخدمة (DoS) .

✓ إدارة بيئة العمل والاعتماديات (Environment & Dependency

Management) يعتمد Laravel على أداة Composer لإدارة المكتبات،

ويستخدم ملف .env لعزل المتغيرات الحساسة (مثل إعدادات الاتصال بقاعدة البيانات

ومحرك الطوابير). هذا الفصل ساهم في مرونة نقل المشروع بين بيئة التطوير وبيئة الإنتاج بأمان

تام.

❖ الطلب الأول – حماية البيانات المشتركة من التضارب (& Concurrent Access)

:(Data Integrity)

• كيف كان يعمل النظام (قبل التعديل):

- في الوضع الطبيعي، كان النظام يقرأ كمية المخزون ويتحقق منها، ثم يخصمها.
- المشكلة تظهر في أوقات الضغط العالي، كما في السيناريو الآتي:
- لدينا قطعة واحدة فقط من المنتج "X" في المخزون.
- المستخدم A والمستخدم B ضغطا على زر "إتمام الطلب" في نفس الجزء من الثانية.
- طلب المستخدم A يقرأ المخزون ويجده = 1
- طلب B يقرأ المخزون ويجده = 1، وذلك لأن طلب المستخدم A لم يخصم الكمية بعد.
- بالتالي، النظام سمح لكلا الطلبين بالمرور، وتم خصم الكمية مرتين.
- النتيجة الكارثية هي أن المخزون أصبح بالسالب، وقمنا ببيع منتج لا نملكه، مما يسبب كارثة في إدارة المبيعات.

• كيف أصبح يعمل النظام (بعد التعديل):

- تم تطبيق تقنية القفل المتشائم (Pessimistic Locking) باستخدام دالة `lockForUpdate()`، وهذا تفصيله:
- ✓ عندما يصل طلب المستخدم A إلى قاعدة البيانات، تقوم الدالة بقفل الصف الخاص بالمنتج X على مستوى قاعدة البيانات (Row-Level Lock)
- ✓ عندما يحاول طلب المستخدم B قراءة نفس المنتج في نفس اللحظة، سيجد الصف مقفلاً، فيُجبر على الانتظار حتى ينهي المستخدم A عملياته (Transaction) سواء بالنجاح Commit أو الفشل Rollback.
- ✓ بعد انتهاء المستخدم A يدخل المستخدم B ولكن هذه المرة سيقراً المخزون بالرقم

✓ الجديد فيجده 0، عندها سيقوم النظام برفض طلبه مع رسالة "المخزون غير كافٍ"، مما يضمن سلامة البيانات (Data Integrity) بنسبة مئة بالمئة.

• كيف كان يعمل النظام (قبل التعديل):

- في البداية كانت عملية الدفع (Checkout) تمر بعدة خطوات متسلسلة منفصلة:
 - ✓ إنشاء سجل الطلب.
 - ✓ إنشاء تفاصيل الطلب.
 - ✓ خصم الكمية من المخزون.
 - ✓ إفراغ سلة المشتريات.
- لكن، ماذا لو تم إنشاء الطلب وخصم المخزون بنجاح، ولكن حدث انقطاع مفاجئ في الاتصال بقاعدة البيانات أو خطأ برمجي قبل إفراغ سلة المشتريات؟
- النتيجة: النظام سيحتوي على بيانات مشوهة؛ العميل خسر أمواله (أو تم تسجيل الطلب عليه)، المخزون نقص، ولكن السلة لا تزال ممتلئة! وهذا يضرب موثوقية النظام بالكامل.

• كيف أصبح يعمل النظام (بعد التعديل):

- تم استخدام تقنية Transaction Management، والذي يضمن مبدأ الذرية (Atomicity) في قاعدة البيانات، أي ضمان تنفيذ جميع خطوات الدفع ككتلة واحدة (الكل أو لا شيء) لمنع تضارب البيانات.
- تم تغليف عملية الدفع ال Checkout بالكامل داخل معاملة واحدة باستخدام
DB::beginTransaction()
- آلية العمل: قمنا بإخبار قاعدة البيانات أن تقوم بتنفيذ أوامر الإنشاء والخصم في الذاكرة المؤقتة فقط، ولا تقومي بحفظها بشكل نهائي إلا عندما أعطيك أمر DB::commit()

■ في حال حدوث أي خطأ في أي خطوة بسبب نقص المخزون، أو خطأ في السيرفر أو غير ذلك، سيقوم النظام بتنفيذ أمر DB::rollBack()

■ النتيجة: يتم التراجع عن جميع الخطوات السابقة فوراً وكأنها لم تكن، لتعود قاعدة البيانات إلى حالتها الأصلية النظيفة قبل بدء الطلب، مما يضمن سلامة وموثوقية البيانات.

❖ الطلب الثاني – إدارة الموارد الحاسوبية (Resource Management &)

:(Capacity Control

● كيف كان يعمل النظام (قبل التعديل):

- كانت الإعدادات الافتراضية لـ Apache Configuration – Laravel تسمح للمستخدم بإرسال عدد غير محدود من الطلبات في الثانية الواحدة. فكان النظام يفتقر إلى آلية تسمى التحكم في التدفق Flow Control.
- المشكلة تظهر هي أنه إذا قام شخص بتكرار طلب "إتمام الشراء" مئات المرات بسرعة فائقة، أو عند دخول عدد كبير من المستخدمين في نفس اللحظة لإصدار فواتير أو إتمام عمليات شراء، كان السيرفر يحاول معالجة جميع الطلبات "دفعة واحدة" بشكل متوازي غير محدود.
- يؤدي ذلك إلى استهلاك كامل للذاكرة RAM وضغط 100% على المعالج، مما يسبب تباطؤاً حاداً في الاستجابة لكل المستخدمين، وفي النهاية ينهار السيرفر Out of Memory Error يتوقف النظام عن العمل تماماً.

● كيف أصبح يعمل النظام (بعد التعديل):

- الهدف: موازنة استهلاك موارد الخادم (CPU, RAM, I/O) لضمان تقديم أسرع استجابة ممكنة ومنع انهيار النظام ومنع استنزاف موارد السيرفر نتيجة الطلبات المتكررة أو العشوائية.
- لتحقيق ما سبق، تم تطبيق استراتيجية "التحكم في السعة" من خلال ثلاثة مستويات، والتي هي:

1. تطبيق تقنية الـ Rate Limiting (على مستوى الكود / Laravel):

✓ تم إعداد "محدد طلبات" برمجياً (Throttle Middleware) على الـ API الخاصة

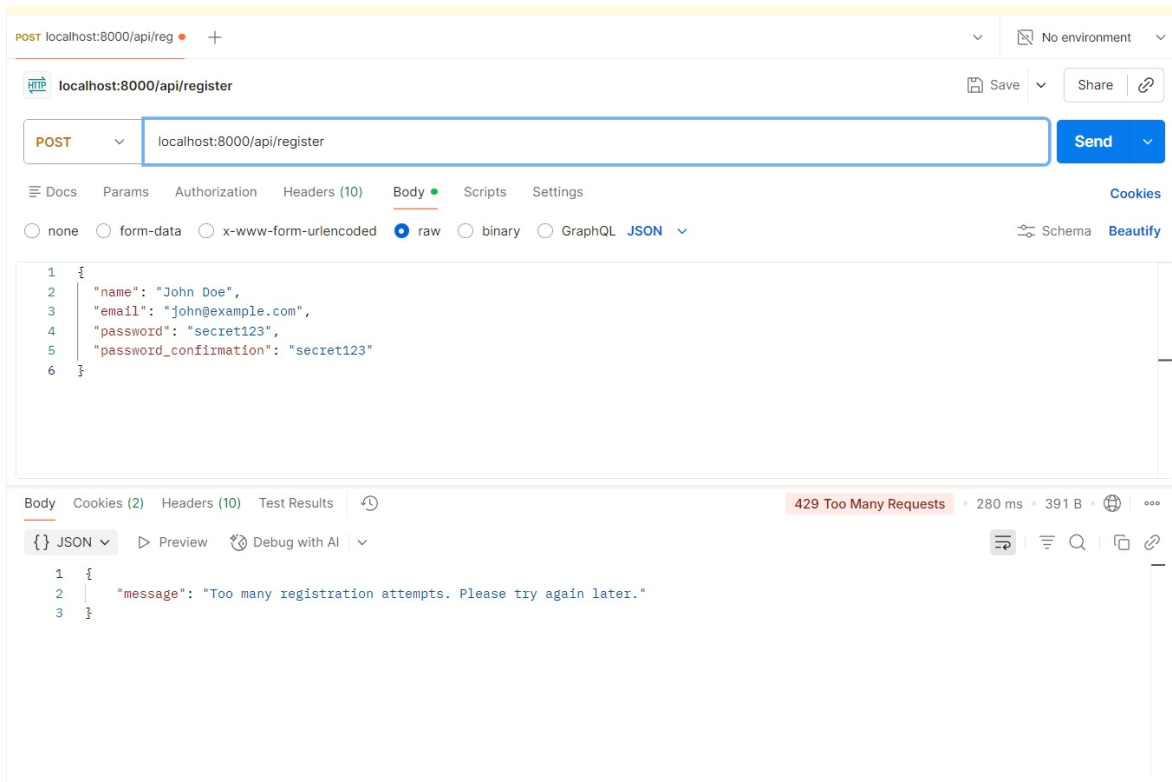
✓ بالدفع والسلة وتسجيل دخول وغيرها.

✓ آلية العمل: قمنا بتحديد سقف معين (مثلاً 10 طلبات في الدقيقة لكل مستخدم). إذا

تجاوز المستخدم هذا الحد، سيقوم النظام برفض الطلب فوراً وإرجاع كود

✓ HTTP 429 (Too Many Requests) دون إشغال قاعدة البيانات أو موارد

المعالج بالعمليات الثقيلة.



هنا نحن قمنا بعمل limit لل بحيث إذا حدا طلب ال api أكثر من 3 مرات خلال مدة

متقاربة جداً ما بخليه يفوت apis

2. إدارة العمليات المتوازية عبر تقنية ال Queue Worker:

■ بدلاً من تنفيذ 100 عملية توليد فاتورة في نفس اللحظة (مما يسبب انهيار السيرفر)، استخدمنا

■ نظام الطوابير Queues.

■ آلية العمل: النظام الآن يستقبل 100 طلب شراء في ثانية واحدة، لكنه يقوم بـ "جدولة"

العمليات الثقيلة (توليد الفواتير) في الخلفية. لذلك قمنا بتشغيل عدد محدد من ال Workers

(مثلاً 2 أو 3 عمال فقط). هذا يعني أن النظام يعالج 3 فواتير فقط في نفس اللحظة، والباقي ينتظر في الطابور.

- النتيجة: السيرفر يعمل بانتظام وبسرعة ثابتة دون أن يقترب من مرحلة الانهيار، لأننا تحكمنا في "عدد العمليات المتوازية".

3. من خلال تعديل ملف `httpd.conf - php.ini` وتفعيل محرك `OpCache` في `Xampp`:

- تعديل ذاكرة التشغيل `memory_limit`. بحيث جعلنا القيمة تساوي 512 MB. بحيث أن هذا السطر يحدد أقصى مقدار من ذاكرة الوصول العشوائي (RAM) يمكن لعملية PHP واحدة استهلاكها. بما أن عملية توليد الفواتير تتطلب معالجة ملفات PDF وتنسيقات معقدة، فإن رفع الذاكرة يضمن عدم توقف النظام فجأة بطلب خطأ ويسمح بمعالجة كميات كبيرة من البيانات داخل الذاكرة بسرعة.
- تعديل وقت تشغيل السكريبت `max_execution_time` وجعلناه 300. بحيث يحدد المدة الزمنية (بالثواني) التي يُسمح فيها للسكريبت بالعمل قبل أن يقوم السيرفر بإيقافه إجبارياً. بحيث رفعنا الوقت إلى 5 دقائق مما يضمن اكتمال العمليات الثقيلة (مثل بناء فاتورة
- ضخمة أو معالجة طابور مهام) دون انقطاع، وفي نفس الوقت يحمي السيرفر من العمليات التي قد تدخل في حلقة مفرغة عبر إنهاؤها بعد هذه المدة.
- تم تحديد حجم رفع البيانات من خلال المتغيرين: `post_max_size` - `upload_max_filesize` وجعلناه 100 MB. هذا الأمر يضمن قدرة النظام على التعامل مع رفع أو تحميل ملفات الفواتير والتقارير الكبيرة، ويمنع السيرفر من رفض الطلبات التي تحتوي على بيانات ضخمة.
- تم التعديل على المتغيرات الآتية: `opcache.enable = 1` - `opcache.memory_consumption = 128`. هذان الخياران يقوموا بتخزين الكود البرمجي المترجم في الرام. فبدلاً من قيام السيرفر بقراءة وترجمة ملفات الكود عند كل طلب (Request)، فإنه يقرأ النسخة المترجمة مسبقاً من الذاكرة. هذا يقلل استهلاك المعالج بشكل

هائل ويزيد من سرعة استجابة الموقع تحت الضغط العالي. كذلك تم تفعيل خيار `opcache.fast_shutdown`. ففي الحالة العادية، يتم تنظيف الذاكرة قطعة قطعة مما يأخذ مدة زمنية، لكن بعد تفعيل هذا الخيار يتم تحرير كتلة الذاكرة بالكامل دفعة واحدة بمجرد انتهاء العملية، مما يوفر وقتاً زمنياً.

■ قمنا بتحديد عدد المسارات أو القنوات التي يفتحها Apache من خلال المتغير: `httpd-mpm.conf` لمعالجة طلبات المستخدمين في نفس الوقت. فزيادة هذا الرقم نرفع من قدرة السيرفر على استقبال عدد أكبر من المستخدمين المتزامنين، مما يمنع رسالة تخبر المستخدمين بانشغال السيرفر.

■ قمنا بتفعيل خيار `Keep-Alive` والذي يتيح بقاء اتصال ال `TCP` مفتوحاً لإرسال عدة طلبات عبر نفس الاتصال، الأمر الذي يقلل من الجهد الضائع في "فتح وإغلاق" الاتصالات المتكررة، مما يسرع تحميل الصفحات التي تحتوي على صور وفواتير متعددة ويخفف الضغط على موارد الشبكة في الجهاز.

❖ الطلب الثالث: المعالجة غير المتزامنة (`Asynchronous Queues`):

● كيف كان يعمل النظام (قبل التعديل):

- كانت عملية إرسال فاتورة الطلب تتم في نفس اللحظة التي يتم فيها معالجة الدفع. بحيث أن خط سير العملية ينتقل عبر الخطوات التالية:
 - ✓ يضغط المستخدم "تأكيد الطلب".
 - ✓ النظام يخصم المخزون.
 - ✓ يبدأ النظام في استدعاء مكتبات توليد ال `PDF` وتنسيق بيانات الفاتورة وحفظها في السيرفر
 - ✓ يعود بالرد للمستخدم.
- المشكلة تكمن في أن عملية تحويل ملفات ال `PDF` هي عملية "ثقيلة" تستهلك الكثير من الموارد، فإذا كان لدى المستخدم 20 منتجاً في الطلب، فقد تستغرق عملية بناء ملف الفاتورة وتنسيقه من 3 إلى 7 ثوانٍ.

- النتيجة: المستخدم يشعر ببطء وتجمد في المتصفح أثناء هذه الثواني، مما يقلل من
- جودة تجربة المستخدم، ويزيد من استهلاك موارد السيرفر في وقت واحد.

● كيف أصبح يعمل النظام (بعد التعديل):

- تم تحسين سرعة استجابة النظام من خلال عزل العمليات التي تستهلك موارد المعالج والذاكرة عن عملية الشراء المباشرة.

- تم تحقيق ذلك من خلال استخدام أمر `SendOrderInvoice::dispatch($order)` لترحيل عملية معالجة الفاتورة إلى الطابور الخلفي.

- آلية العمل: بمجرد نجاح الـ `Transaction` في قاعدة البيانات، يرمي النظام "مهمة معالجة الفاتورة" في الطابور ويحرر اتصال المستخدم فوراً.

■ لماذا استخدمنا الـ Queue هنا؟

1. تجنب المهلة الزمنية: في حال كانت الفاتورة ضخمة جداً، قد يتوقف السيرفر عن العمل، لكن الـ Queue لا يتأثر بهذا النوع من القيود.
 2. الاستقرار: إذا حدث خطأ أثناء توليد الـ PDF (نقص في الخطوط أو مشكلة في التنسيق)، لن يفشل طلب الشراء؛ بل سيبقى الطلب ناجحاً، ويقوم الـ Worker بإعادة محاولة توليد الفاتورة لاحقاً.
 3. تحسين الأداء: يستطيع السيرفر معالجة مئات طلبات الشراء في الثانية الواحدة، بينما يتفرغ العمال في الخلفية لتوليد الفواتير واحداً تلو الآخر دون التأثير على سرعة تصفح الموقع.
- بما أن النظام يعتمد على قاعدة البيانات كـ `Driver` لإدارة الطوابير (`QUEUE_CONNECTION=database`)، فقد تم إعداد البيئة تقنياً كالتالي:

1. آلية التخزين (Storage Strategy):

- ✓ تم إنشاء جدول مخصص في قاعدة البيانات يسمى `jobs`. هذا الجدول يعمل كـ "خزان" للمهام؛ فعند استدعاء الفاتورة، لا يتم تنفيذ الكود برمجياً في تلك اللحظة، بل يتم تحويل

✓ كائن الفاتورة (Order Object) إلى بيانات نصية (Serialized) وتخزينها في هذا الجدول مع تحديد الأولوية ووقت التنفيذ.

2. آلية التنفيذ (Worker Execution):

✓ لضمان عمل النظام، يتم تشغيل عملية مراقبة Background Process باستخدام الأمر `php artisan queue:work` بحيث يقوم هذا الأمر بفحص جدول `jobs` باستمرار.

✓ بمجرد اكتشاف مهمة جديدة (فاتورة بانتظار التوليد)، يقوم بسحبها، معالجتها، ثم حذفها ✓ من الجدول عند النجاح.

3. معالجة الأخطاء (Error Handling):

✓ من أهم ميزات هذا الإعداد هو جدول `failed_jobs`. ففي حال حدث خطأ تقني أثناء توليد ملف الفاتورة (مثل نقص في الذاكرة أو خطأ في مسار الحفظ)، لن تضيع الفاتورة؛ بل سيقوم النظام بنقل المهمة إلى هذا الجدول، مما يتيح لي كمطور مراجعة الخطأ وإعادة تشغيل المهمة لاحقاً دون إزعاج المستخدم.

❖ الطلب الرابع – معالجة البيانات الضخمة على دفعات (Batch Processing)

• كيف كان يعمل النظام (قبل التعديل):

- كان من الممكن تنفيذ الجرد اليومي للمبيعات بطريقة مباشرة عبر تحميل جميع الطلبات المكتملة الخاصة باليوم مع جميع عناصرها (Order Items) دفعة واحدة داخل الذاكرة، ثم إجراء عمليات التجميع والحساب داخل PHP.
- المشكلة تظهر عندما يبدأ حجم البيانات بالازدياد، كما في السيناريو الآتي:
 - ✓ متجر التجارة الإلكترونية يحتوي على آلاف الطلبات يومياً.
 - ✓ كل طلب يحتوي على عدد كبير من المنتجات.
 - ✓ النظام يقوم بتحميل جميع الطلبات والعناصر دفعة واحدة إلى الذاكرة.
 - ✓ ثم يبدأ بمعالجة البيانات وتجميع الإيرادات والكميات المباعة داخل التطبيق.

■ النتيجة:

✓ استهلاك ضخم للذاكرة.

✓ ارتفاع زمن التنفيذ بشكل كبير.

✓ زيادة احتمالية توقف العملية بسبب أخطاء مثل: Out Of Memory -

Maximum Execution Time Exceeded

■ بالإضافة إلى ذلك، فإن تنفيذ هذه العملية داخل طلب HTTP عادي للمستخدم يؤدي إلى بطء شديد في استجابة النظام، مما يؤثر سلباً على تجربة الاستخدام واستقرار الخادم تحت الضغط العالي .

● كيف أصبح يعمل النظام (بعد التعديل):

■ تم تصميم نظام الجرد اليومي بطريقة تعتمد على المعالجة الخلفية للبيانات باستخدام تقنية ال Batch Processing وذلك لمعالجة البيانات على دفعات صغيرة ومنظمة دون استنزاف موارد السيرفر.

■ تم تنفيذ الجرد اليومي كمهمة خلفية (Background Job) تعمل خارج دورة طلب

المستخدم باستخدام Laravel Queues

■ آلية العمل:

✓ عند تشغيل عملية الجرد، لا يقوم النظام بتحميل جميع الطلبات دفعة واحدة. بدلاً من ذلك،

يستخدم الدالة: chunkById(500) لقراءة البيانات على دفعات صغيرة مكونة من

500 سجل فقط في كل مرة.

✓ بعد الانتهاء من معالجة الدفعة الحالية يتم تحرير الذاكرة ثم الانتقال للدفعة التالية.

✓ بهذه الطريقة، يبقى استهلاك الذاكرة ثابتاً تقريباً مهما ازداد عدد الطلبات داخل قاعدة

البيانات.

■ كيف أصبح النظام يحسب التقارير اليومية؟

✓ تم إنشاء جدول مخصص للملخص اليومي العام يسمى: `daily_sales_summaries` بحيث يقوم هذا الجدول بتخزين ما يلي:

1. عدد الطلبات اليومية.

2. عدد القطع المباعة.

3. إجمالي الإيرادات.

✓ كما تم إنشاء جدول آخر يسمى: `daily_product_sales` لتخزين:

1. الكمية المباعة لكل منتج.

2. الإيراد الخاص بكل منتج.

3. تاريخ البيع.

■ كيف تعمل Job الخاصة بالجرد اليومي؟

✓ تستقبل الـ Job تاريخ اليوم المطلوب إجراء الجرد له، ثم يتم تحويله إلى:

1. `startOfDay`

2. `endOfDay`

✓ وذلك لضمان احتساب الطلبات المكتملة ضمن ذلك اليوم فقط.

✓ بعد ذلك يتم تنفيذ الاستعلام على الطلبات المكتملة فقط. بحيث تبدأ عمليات القراءة على

دفعات باستخدام: `chunkById(500)`

✓ بحيث أن عمليات التجميع لا تحسب يدوياً داخل PHP، بل نعالجها من خلال قاعدة

البيانات، لأن قاعدة البيانات مصممة أساساً لمعالجة العمليات التجميعية الضخمة بكفاءة عالية.

■ كيف نستعلم عن التقرير عن طريق الـ Tirminal ؟

– `php artisan sales:daily-summary` --> شغل التقرير على تاريخ اليوم الحالي.

– `php artisan sales:daily-summary 2026-05-10` --> شغل التقرير على

تاريخ محدد بصيغة Y-m-d .

– `php artisan sales:daily-summary --yesterday`

php artisan sales:daily-summary 2026-05-10 --sync -

ينفذ نفس التقرير مباشرة داخل نفس العملية بدون Queue

■ كيف ساهم Laravel في تحقيق هذا الحل؟

✓ وقرّ Laravel بنية جاهزة لمعالجة المهام الخلفية عبر والتي هي:

1. ShouldQueue

2. dispatch

✓ كما وقرّ Laravel Scheduler لتنفيذ الجرد بشكل يومي تلقائي عند الساعة 01:00

صباحاً دون الحاجة إلى سكريبتات خارجية معقدة.

✓ كما وفر Artisan Commands لتشغيل الجرد يدوياً لتاريخ محدد أو لليوم السابق.

■ لماذا يعتبر هذا التصميم احترافياً؟

✓ يحافظ على سرعة النظام، لأن عملية الجرد لا تعمل داخل طلب المستخدم المباشر

HTTP Request.

✓ يمنع استنزاف الذاكرة لأن chunkById(500) يمنع تحميل جميع البيانات دفعة واحدة.

✓ يستفيد من قوة قاعدة البيانات في عمليات التجميع داخل قاعدة البيانات دون الحاجة

لتعطيل التطبيق في حسابها.

✓ يدعم التوسع المستقبلي، إذ يمكن للنظام معالجة ملايين السجلات تدريجياً دون انهيار الخادم

أو التأثير على المستخدمين الحاليين.

■ وهذه الصور تثبت لنا عمل Asynchronous Queues مع جرد التقارير في

الخلفية :

حيث تم استخدام :

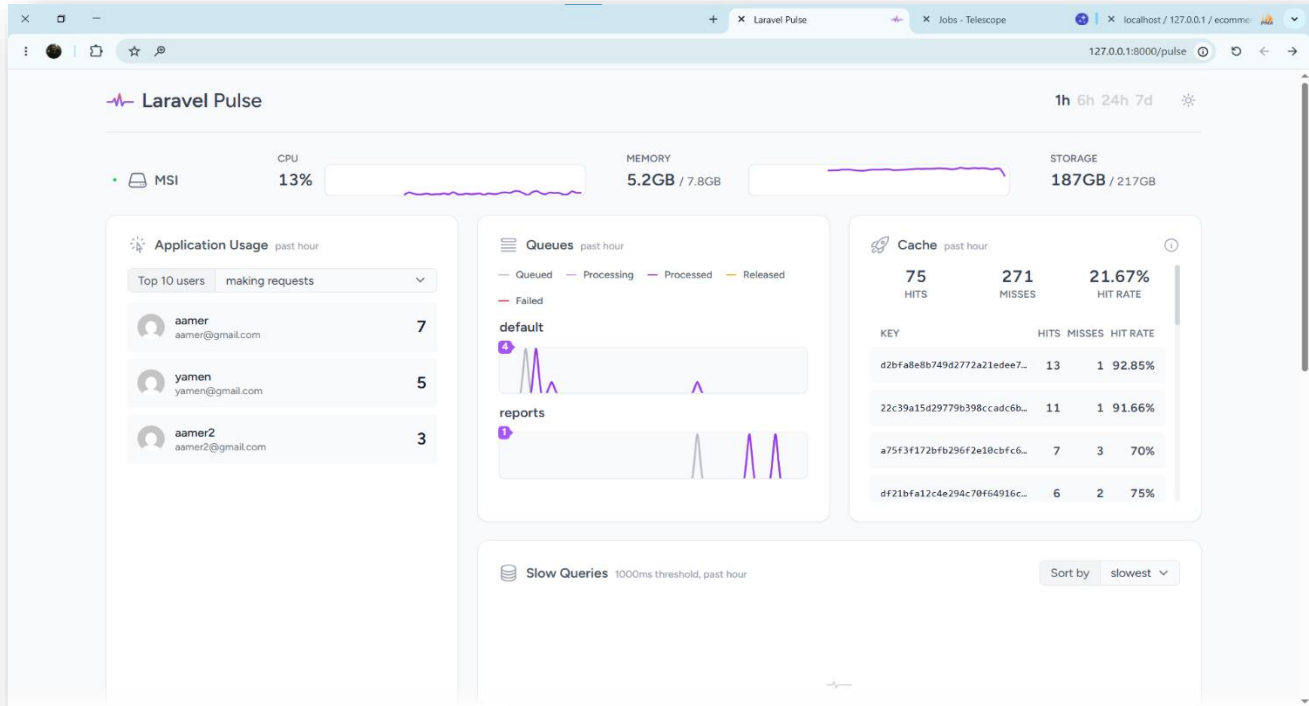
Laravel Telescope: استخدمت كبيئة تطوير مساعدة لتتبع تفاصيل الطلبات

(Requests)، استعلامات قواعد البيانات (Queries)، الأخطاء (Exceptions)،

والوظائف الخلفية (Jobs).

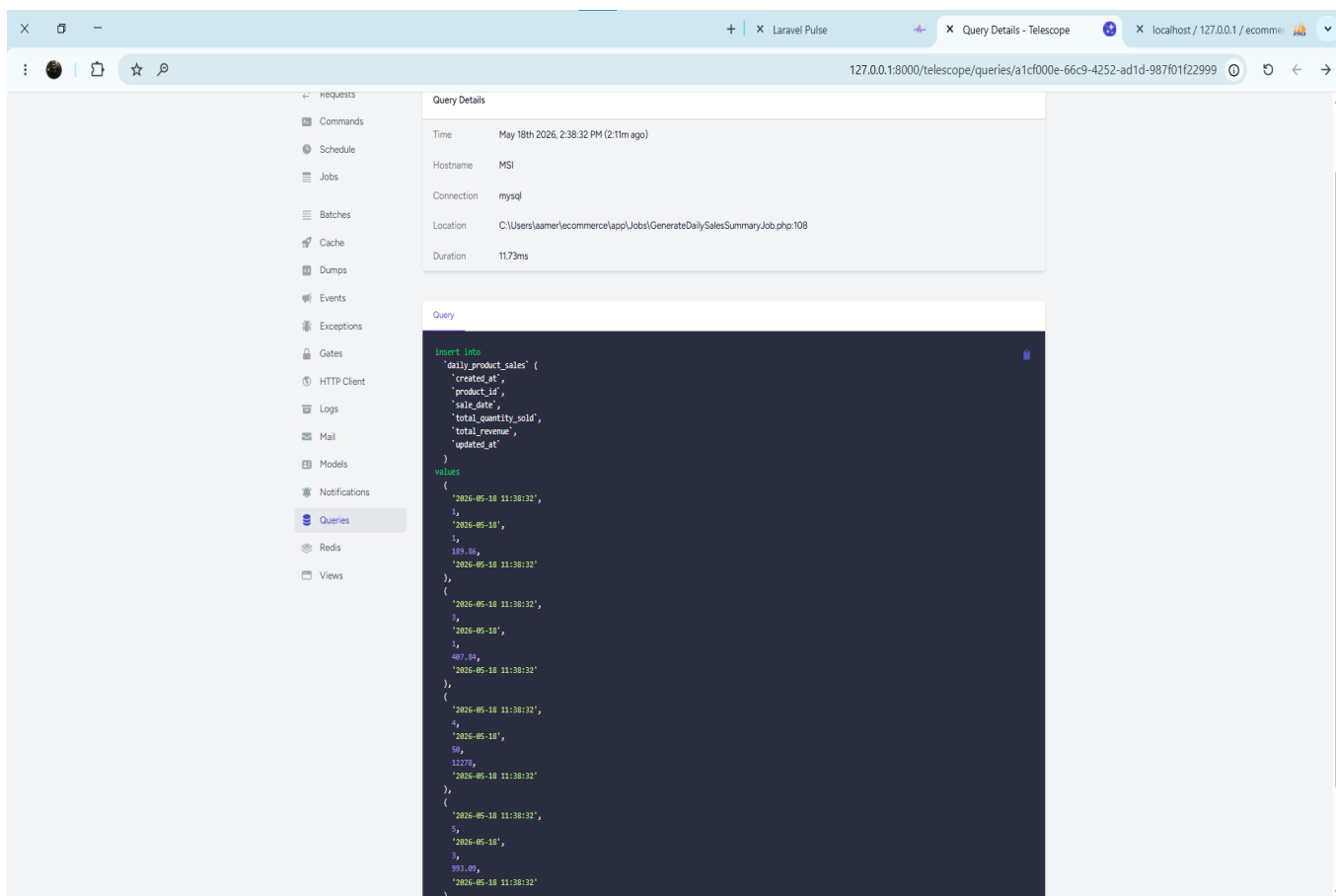
Laravel Pulse: استخدمت لمراقبة أداء التطبيق في الوقت الفعلي (Real-time)

(time)، وتحديد الاختناقات (Bottlenecks)، وتتبع استهلاك المعالج والذاكرة، ومعرفة الاستعلامات الأكثر بطئاً.



The screenshot displays the Laravel Telescope - Jobs view. The left sidebar shows a navigation menu with options: Requests, Commands, Schedule, Jobs (selected), Batches, Cache, Dumps, Events, Exceptions, Gates, HTTP Client, Logs, Mail, Models, and Notifications. The main content area shows a list of jobs with columns: Job, Status, and Happened. The jobs are listed in a table with a search bar at the top right.

Job	Status	Happened
App\Jobs\GenerateDailySalesSummaryJob Connection: database Queue: reports	processed	1:11m ago
App\Jobs\GenerateDailySalesSummaryJob Connection: database Queue: reports	processed	16m ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	16m ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	44m ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	1h ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	1h ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	1h ago
App\Jobs\SendOrderInvoice Connection: database Queue: default	processed	1h ago



❖ الطلب الخامس – توزيع الأحمال (Load Distribution):

- كيف كان يعمل النظام (قبل التعديل): كانت عملية استقبال طلبات المتسوقين وتصفح المنتجات وإتمام عمليات الشراء تتم عبر خادم ويب واحد مستقل (Single Server Instance) مستضاف على منفذ واحد في بيئة العمل. بحيث أن خط سير العملية كان ينتقل عبر الخطوات التالية:

- يدخل المستخدم إلى رابط المتجر الإلكتروني.
- يستقبل الخادم الوحيد الطلب، ويبدأ بمعالجة الاستعلامات البرمجية وعرض المنتجات.
- ينتظر المستخدم استجابة هذا الخادم بالكامل ليتمكن من إتمام التصفح أو الشراء.

- **المشكلة:** تكمن المشكلة في أن الخادم الواحد يمتلك سعة محددة من الذاكرة العشوائية وقدرة المعالج، فإذا تزايد عدد الزوار في نفس اللحظة (كما يحدث في مواسم التخفيضات أو العروض الكبرى)، سيتكدس الضغط على هذا الخادم الفردي، مما يجعله عاجزاً عن معالجة الطلبات المتزامنة.
- **النتيجة:** يواجه المستخدمون بطئاً شديداً وتجمداً في استجابة المتجر، وقد يتوقف الموقع تماماً عن العمل وتظهر أخطاء السيرفر الشهيرة، مما يتسبب في خسارة المبيعات وتراجع جودة تجربة المستخدم، فضلاً عن خطورة وجود "نقطة فشل واحدة" (Single Point of Failure) بحيث لو تعطل السيرفر انهار المتجر بالكامل.
- **كيف أصبح يعمل النظام (بعد التعديل):** تم رفع كفاءة النظام وقدرته الاستيعابية من خلال حماية المتجر من الانهيار عبر بناء بنية تحتية تعتمد على تعدد الخوادم Multi-Instance Architecture وعزل نظام استقبال الطلبات عن نظام المعالجة الفردي. وقد تم تحقيق ذلك من خلال تشغيل نسختين مستقلتين من مشروع المتجر لتعمل كخوادم معالجة خلفية وفق ما يلي:

▪ Server 1 على المنفذ 8001

▪ Server 2 على المنفذ 8002

مع تنصيب خادم Nginx ليعمل كموزع أحمال رئيسي Reverse Proxy Load Balancer يستمع على المنفذ 80 الرئيسي.

- **آلية العمل:** بمجرد دخول أي متسوق للمتجر عبر الرابط العام، يستقبل خادم Nginx الطلب أولاً عبر المنفذ 80، وبدلاً من معالجته في مكان واحد، يقوم بتوجيه حركة المرور وتوزيعها خلف الكواليس فوراً وبتناوب ذكي إلى أحد الخوادم المتاحة (8001 أو 8002)، مما يحرر المتصفح ويمنع تكدس الطلبات.

- لماذا استخدمنا الـ Load Balancer وخوارزمية Round Robin هنا؟

1. استمرارية الخدمة – **High Availability**: في حال تعطل أحد خوادم المتجر تماماً أو توقفه

المفاجئ عن الاستجابة، يقوم الموزع تلقائياً بتحويل كافة حركة المرور إلى الخادم الآخر النشط، مما يضمن بقاء المتجر متاحاً للزبائن دون أي انقطاع.

2. التوزيع العادل للضغط: فالخوارزمية المستخدمة والتي هي Round Robin تضمن توزيع

الطلبات بالتساوي (طلب للخادم الأول يليه طلب للخادم الثاني)، وهو الحل التقني الأمثل بما أن الخوادم تمتلك نفس المواصفات العتادية (CPU/RAM) على نفس الجهاز.

3. قابلية التوسع المستقبلي – **Scalability**: يتيح هذا التصميم للمتجر إمكانية إضافة خوادم

جديدة بكل سهولة (خادم ثالث ورابع) عند نمو حجم التجارة وزيادة عدد الزوار مستقبلاً، دون الحاجة لتغيير الكود البرمجي الأساسي للمشروع.

• الإعداد التقني للبيئة: بما أن النظام يعتمد على خادم Nginx كموزع أحمال رئيسي لإدارة حركة المرور، فقد تم إعداد البيئة تقنياً كالتالي:

4. آلية التوجيه – **Upstream Strategy**: تم إنشاء كتلة برمجية مخصصة في ملف إعدادات

Nginx تسمى upstream وتحديد العناوين المحلية للخوادم الخلفية للمتجر

(127.0.0.1:8001 و 127.0.0.1:8002) ليعتمدها النظام كخزان رئيسي للخوادم

النشطة المتاحة لتلقي الطلبات.

5. آلية التمرير – **Proxy Configuration**: تم ضبط الإعدادات البرمجية لتمرير الهيدرز

والبيانات الحقيقية للزائر مثل IP المستخدم الحقيقي وال Host وال Protocol إلى الخوادم

الخلفية عبر أوامر proxy_set_header لضمان دقة التقارير البرمجية والأمنية وتتبع

حركات الشراء داخل لارافيل بشكل صحيح.

6. إدارة الجلسات وسلة المشتريات – **Session Strategy**: لحماية بيانات المتسوقين من

الضياع أثناء التنقل بين الخوادم، تم ضبط برمجيات الجلسات في لارافيل لتعمل عبر خزان وسيط

مركزي مشترك Database) بدلاً من التخزين المحلي التقليدي في ملفات السيرفر الفردي (file driver)، مما يضمن بقاء سلة المشتريات وحالة تسجيل الدخول للزبون ثابتة ومستقرة تماماً.

- التعديل البرمجي لتمييز الخوادم: تم التعديل على ملف web.php لإتاحة القدرة على تمكين فحص النظام برمجياً والتأكد من هوية الخادم الذي يخدم الطلب الحالي، بحيث تم تعديل المسار الرئيسي في الملف ليعترض الطلب ويجلب رقم المنفذ (Port) الخاص بالسيرفر النشط حالياً، وذلك عبر الكود التالي:

```
Route::get('/', function () { Route::get('/', function () {
    // جلب رقم المنفذ الحقيقي الذي يعالج الطلب الحالي من السيرفر
    $currentPort = Request::server('SERVER_PORT');

    return "
    <div style='text-align: center; margin-top: 15%; font-family: Arial, sans-serif;'>
        <h1>مشروع محاكاة توزيع الأحمال (Load Balancing)</h1>
        <p style='font-size: 1.5rem;'>
            الطلب الحالي يتم معالجته بواسطة الخادم ذو المنفذ:
            <strong style='color: #e53e3e;'>{$currentPort}</strong>
        </p>
        <p>[ قم بتحديث الصفحة أو استخدام التصفح الخفي لمشاهدة التغير ]</p>
    </div>
    ";
});
```

- آلية اختبار ومحاكاة الميزة: يمكن التحقق واختبار المتطلب عملياً عبر تتبع الخطوات التالية:

1. تشغيل الخادم الأول على المنفذ 8001 والخادم الثاني على المنفذ 8002 عبر الترمينال، مع التأكد من تشغيل خادم Nginx كموزع أحمال رئيسي.

2. فتح المتصفح على الرابط <http://localhost> والانتقال للرابط الموحد للمتجر بدون تحديد أي منفذ في الرابط.

3. سنلاحظ أن الصفحة تفتح مباشرة وتُظهر أن السيرفر المستجيب هو (مثلاً 8001).

4. عند القيام بعمل تحديث قسري للصفحة سنلاحظ فوراً قفز النظام وتغير المنفذ المعروض إلى (8002) بالتناوب التام.

5. بالتوازي مع المتصفح، يمكن مراقبة واجهتي الترمينال (Terminal) الخاصتين بالخوادم، حيث ستظهر سجلات الطلبات (Logs) تتوارد في الشاشة الأولى ثم الشاشة الثانية بالتناوب الحقيقي مع كل نقرة، مما يؤكد تفعيل تقنية الـ Load Balancing بنجاح.

● لماذا يعتبر هذا التصميم احترافياً؟

1. يحافظ على سرعة استجابة المتجر الإلكتروني وسلاستها حتى تحت الضغط العالي وخلال حملات التسوق الكبرى.

2. يقضي تماماً على مشكلة "نقطة الفشل الواحدة" ويجعل نظام المتجر مقاوماً للأعطال الفنية المفاجئة.

3. يمنع استنزاف موارد سيرفر واحد من خلال توزيع العبء الحسابي لعمليات التصفح والشراء بالتساوي وبكفاءة متناهية.

4. يدعم التوسع المستقبلي الفوري، إذ يمكن للمتجر استيعاب ملايين العمليات المتزامنة تدريجياً بمجرد ربط خوادم إضافية بسلسلة ودون الحاجة لتغيير بنية التطبيق.

التعديل الذي طبق على ملف الإعدادات لـ nginx :

```
events { events
    worker_connections 1024;
}

http {

    # تحديد الخوادم الوهمية التي ستستقبل
    # الطلبات (مشروع لارافيل الخاص بك)
    upstream my_laravel_project {
        server 127.0.0.1:8001;
        server 127.0.0.1:8002;
    }

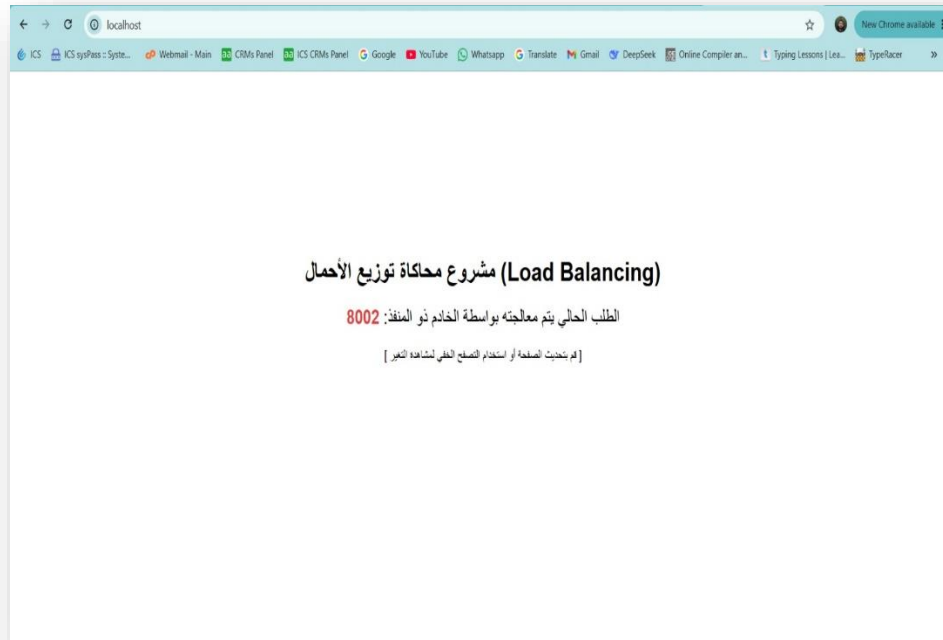
    server {
        listen 80 # المنفذ الرئيسي لـ 80 ;
        server_name localhost;

        location / {

            # توجيه حركة المرور إلى مجموعة
            # السيرفرات أعلاه
            proxy_pass
            http://my_laravel_project;

            # تمرير البيانات الحقيقية للطلب
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP
            $remote_addr;
            proxy_set_header X-Forwarded-For
            $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-
            Proto $scheme;
        }
    }
}
```

وهذه الصور عن تطبيق هذا المفهوم :

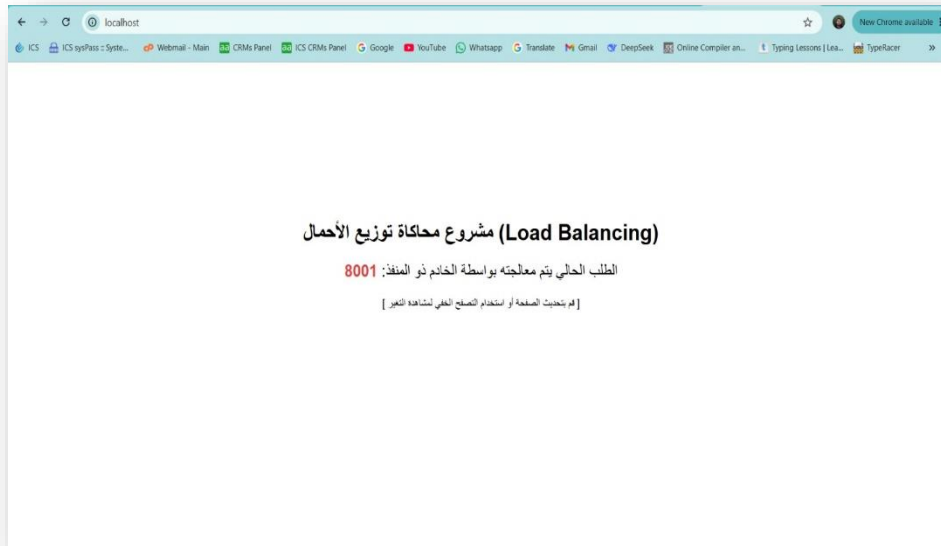


```
Problems  Output  Debug Console  Terminal
PS C:\Users\USER\Desktop\Ecommerce> php artisan serve --port=8002

INFO Server running on [http://127.0.0.1:8002].

Press Ctrl+C to stop the server

2026-05-17 19:24:15 ..... ~ 2s
2026-05-17 19:24:17 /favicon.ico ..... ~ 0s
2026-05-17 19:24:21 ..... ~ 1s
2026-05-17 19:24:22 /favicon.ico ..... ~ 0s
2026-05-17 19:24:23 ..... ~ 0s
2026-05-17 19:24:23 /favicon.ico ..... ~ 0s
2026-05-17 19:24:23 ..... ~ 0s
```



```
Problems Output Debug Console Terminal
PS C:\Users\USER\Desktop\Ecommerce> php artisan serve --port=8001

INFO Server running on [http://127.0.0.1:8001].

Press Ctrl+C to stop the server

2026-05-17 19:24:59 ..... ~ 0s
2026-05-17 19:25:00 ..... ~ 0s
2026-05-17 19:25:00 ..... ~ 1s
2026-05-17 19:25:01 ..... ~ 0s
2026-05-17 19:25:02 ..... ~ 0s
2026-05-17 19:25:02 ..... ~ 0s
2026-05-17 19:25:03 ..... ~ 0s
```